

INFORME TÉCNICO — ES2

Implementación de Software Orientado a Objetos en Python

Caso EcoTech Solutions

TI3021 · Programación Orientada a Objeto Seguro

Unidad 2 · Implementación y persistencia segura

Nombre: Benjamin Olivares Mella

RUT: 20.777.250-K

Carrera: Analista Programador

Docente: Nilson A. Guerra Cabeza

Fecha: 22 de octubre de 2026

Responsable: Benjamin Olivares Mella

Índice de Contenidos

1. Introducción y Alcance de la Solución 3
2. Arquitectura y Organización del Código Fuente 3
2.1 Estructura del Proyecto 3
2.2 Separación de Responsabilidades 3
2.3 Entorno y Ejecución 4
3. Implementación del Modelo Orientado a Objetos 4
3.1 Entidades y Responsabilidades del Sistema 4
3.2 Constructores y Métodos del Dominio 5
3.3 Principios de Programación Orientada a Objetos 6
4. Persistencia y Acceso a los Datos 7
4.1 Conexión y Configuración 7
4.2 Estructura Relacional 8
4.3 Operaciones CRUD 9
4.4 Conversión de Filas en Objetos 10
4.5 Comprobación de Consultas Parametrizadas 11
5. Manejo de Errores y Transacciones 11
5.1 Excepciones del Sistema 11
5.2 Validación de Entradas y Control de Errores 12
5.3 Confirmación y Reversión de Operaciones 13
6. Uso de IA y Revisión del Código 14
6.1 Herramienta y Solicitudes Técnicas 14
6.2 Análisis de Propuestas y Decisiones 14
6.3 Resultados de las Comprobaciones 15
7. Trabajo Individual y Conclusiones 15
7.1 Aportes y Responsabilidad de la Entrega 15
7.2 Conclusiones 15
8. Referencias Bibliográficas 15

1. Introducción y Alcance de la Solución

EcoTech Solutions necesita mantener ordenada la información de sus empleados, departamentos y proyectos. El uso de registros separados dificulta conocer las asignaciones y relacionar las horas trabajadas con cada proyecto. A partir del análisis y del modelo de clases del primer informe, se desarrolló una aplicación en Python para gestionar estos datos y conservarlos en una base de datos.

La solución permite registrar, consultar, actualizar y eliminar empleados, departamentos, proyectos y registros de tiempo. También administra responsables de departamento y asignaciones a proyectos, y genera resúmenes de salarios y horas. El sistema funciona por consola y utiliza SQLite como almacenamiento local.

2. Arquitectura y Organización del Código Fuente

2.1 Estructura del Proyecto

Los archivos se organizan según su responsabilidad. El dominio representa las entidades de EcoTech, los repositorios guardan y recuperan información y main.py reúne las operaciones disponibles en el menú.

```
ecotech_es2/
    dominio/
        persona.py, empleado.py, departamento.py
        proyecto.py, registro_tiempo.py, validaciones.py
        excepciones.py, informe.py
        informe_nomina.py, informe_proyecto.py
    repositorio/
        conexion.py, sistema_repo.py
        empleado_repo.py, departamento_repo.py
        proyecto_repo.py, registro_tiempo_repo.py
    db/01_esquema.sql
    tests/test_sistema.py
    evidencias/
    docs/
    main.py, demo.py, iniciar.cmd
    README.md, requirements.txt, .env.example, .gitignore
    Informe_ES2_EcoTech.docx, Informe_ES2_EcoTech.pdf
```

Los paquetes incluyen __init__.py. El archivo SQL define las tablas y sus restricciones; tests contiene las pruebas de funcionamiento y evidencias conserva sus resultados. La configuración local, la base de datos y el entorno virtual se crean al preparar la aplicación y se excluyen de la entrega.

2.2 Separación de Responsabilidades

Las clases del dominio contienen los datos y las validaciones de empleados, departamentos, proyectos y registros de tiempo. No ejecutan instrucciones SQL. Los repositorios reciben objetos, preparan sus valores y ejecutan las operaciones de almacenamiento. Esta separación evita que una regla del salario dependa de cómo se abre una conexión.

BaseDatos administra la conexión y las transacciones. SistemaEcoTech coordina operaciones que afectan a varias entidades, como asignar un empleado a un proyecto. Cada repositorio contiene sus operaciones CRUD completas y la conversión de sus filas en objetos. Esta organización permite seguir una operación sin buscar su implementación en una clase genérica.

Si se cambia el motor de base de datos, la revisión se concentra en la conexión, el SQL, el esquema y la configuración. Las entidades pueden conservarse mientras se mantengan sus responsabilidades y la

interfaz de los repositorios. La separación reduce la dependencia entre las reglas del sistema y el almacenamiento.

2.3 Entorno y Ejecución

La aplicación requiere Python 3.10 o posterior. Utiliza sqlite3 y otros módulos de la biblioteca estándar, por lo que no necesita paquetes externos. La preparación y ejecución en Windows se resumen en la siguiente tabla.

Paso	Comando o acción	Resultado
1	python -m venv .venv	Crea el entorno de ejecución.
2	.venv\Scripts\activate	Activa el entorno.
3	python -m pip install -r requirements.txt	Comprueba las dependencias declaradas.
4	Copiar .env.example como .env	Configura DB_NOMBRE=datos/ecotech.db.
5	python main.py	Inicializa las tablas y abre el menú.
6	python -m unittest discover -v	Ejecuta las 37 pruebas automáticas.

Se comprobó la ejecución desde una copia del proyecto sin datos locales, utilizando un entorno virtual nuevo. Se creó un empleado en un proceso y se recuperaron sus datos en otro, después de cerrar el programa. Ambos procesos finalizaron correctamente. El registro de esta comprobación está en evidencias/ejecucion_limpia.txt.

3. Implementación del Modelo Orientado a Objetos

3.1 Entidades y Responsabilidades del Sistema

El modelo organiza la información alrededor de Persona, Empleado, Departamento, Proyecto y RegistroTiempo. Se conservan las responsabilidades de identificar a los trabajadores, administrar sus asignaciones y relacionar las horas con el empleado y el proyecto correspondientes. La gerencia se administra mediante el rol del empleado y la relación de responsabilidad con el departamento.

Clase	Archivo y constructor	Responsabilidad
Persona	dominio/persona.py: __init__(nombre, direccion, telefono, email, id)	Mantener y actualizar los datos personales comunes.
Empleado	dominio/empleado.py: __init__(nombre, direccion, telefono, email, fecha_inicio, salario, departamento_id, rol, id)	Representar al trabajador y validar su información laboral.
Departamento	dominio/departamento.py: __init__(nombre, gerente_id, id)	Representar un área y su responsable.
Proyecto	dominio/proyecto.py: __init__(nombre, descripcion, fecha_inicio, id)	Mantener los datos del trabajo desarrollado por la empresa.
RegistroTiempo	dominio/registro_tiempo.py: __init__(empleado_id, proyecto_id, fecha, horas, descripcion, id)	Registrar horas vinculadas con una asignación.
Informe	dominio/informe.py: clase abstracta, sin constructor propio	Definir la operación exportar para los resúmenes.
InformeNomina	dominio/informe_nomina.py: __init__(empleados)	Obtener el total de salarios de los empleados.

Clase	Archivo y constructor	Responsabilidad
InformeProyecto	dominio/informe_proyecto.py: __init__(proyecto, registros)	Obtener el total de horas de un proyecto.

Cada empleado puede pertenecer a un departamento y participar en varios proyectos. A su vez, un proyecto puede tener varios empleados. RegistroTiempo identifica una asignación concreta y conserva fecha, horas y descripción. Los registros se identifican mediante un ID interno; el correo del empleado tiene una restricción de unicidad.

Las operaciones de asignación se coordinan fuera de las entidades para guardar las relaciones en la base de datos. Las clases mantienen sus datos y validaciones; SistemaEcoTech administra los vínculos y las operaciones que requieren varias escrituras.

Responsabilidad del modelo	Operación implementada
Asignar o retirar empleados de un proyecto	SistemaEcoTech.vincular y desvincular guardan la relación empleado_proyecto.
Gestionar al responsable de un departamento	SistemaEcoTech.asignar_gerente y quitar_gerente mantienen la responsabilidad y comprueban el rol.
Asignar o reasignar un empleado a un área	Empleado.asignar_departamento modifica el dato y empleados.actualizar lo conserva.
Registrar y validar las horas trabajadas	RegistroTiempo valida sus datos; el repositorio comprueba el total diario y guarda el registro. SQLite exige una asignación válida.

3.2 Constructores y Métodos del Dominio

El constructor de Empleado recibe los datos personales y laborales. Primero llama al inicializador de Persona y después asigna fecha de inicio, salario, departamento y rol. Las propiedades verifican estos valores durante la creación del objeto. Un empleado nuevo conserva id=None hasta que se confirma su registro en SQLite.

```
def __init__(self, nombre: str, direccion: str, telefono: str, email: str,
             fecha_inicio: str, salario: float, departamento_id: int = None,
             rol: str = 'empleado', id: int = None):
    """Valida también el salario inicial, no solo sus modificaciones."""
    super().__init__(nombre, direccion, telefono, email, id)
    self.fecha_inicio = fecha_inicio
    self.salario = salario
    self.departamento_id = identificador(departamento_id, opcional=True)
    self.rol = rol
```

El método subir_salario comprueba que el aumento sea positivo y asigna el nuevo valor mediante la propiedad salario. De esta forma se revisan tanto el aumento como el resultado antes de modificar el dato interno.

```
def subir_salario(self, aumento: float):
    """Aplica un aumento positivo y valida el resultado."""
    self.salario = self.salario + positivo(aumento, 'Aumento')
```

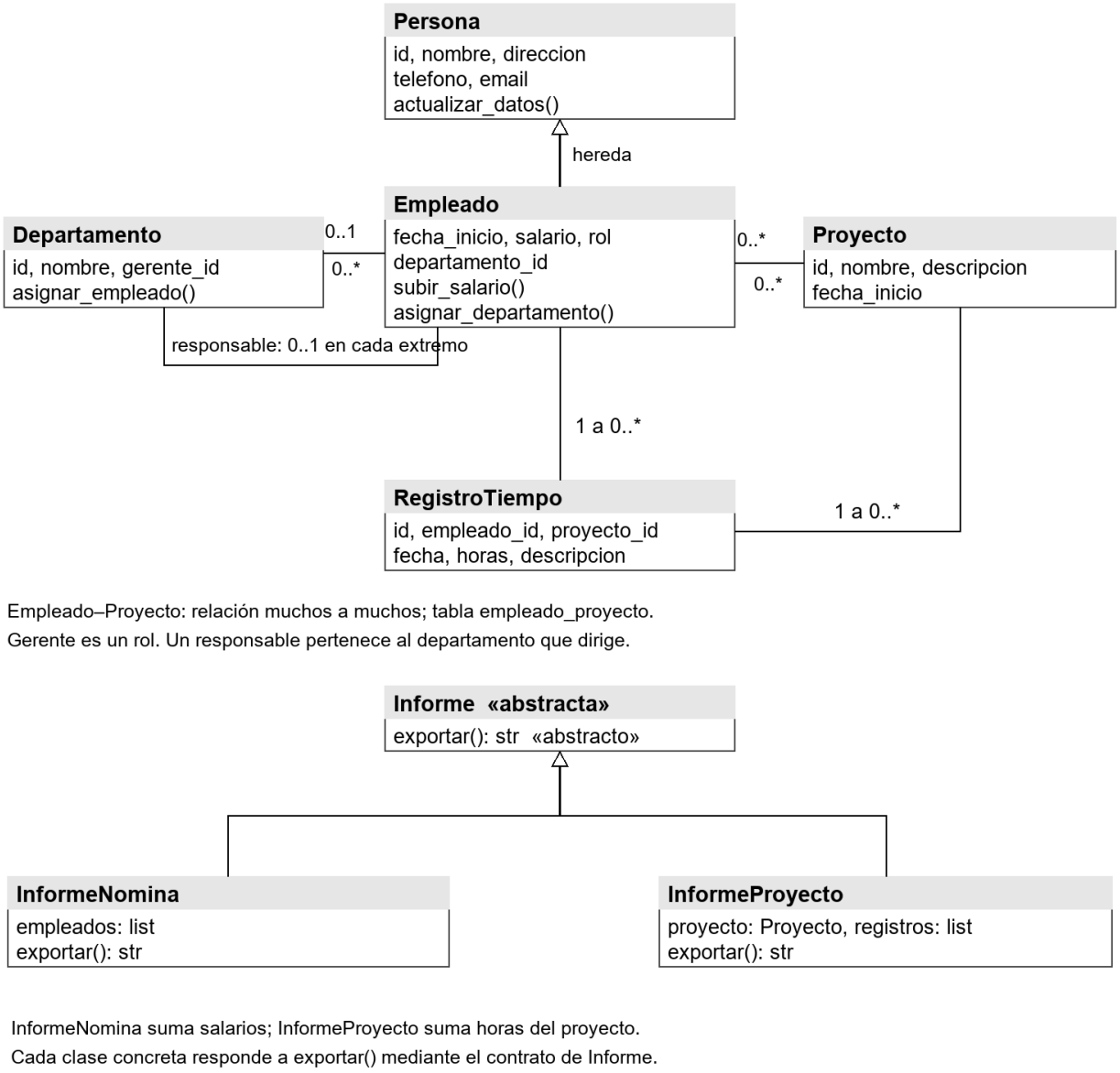


Figura 1. Clases y relaciones implementadas en el sistema de EcoTech Solutions.

3.3 Principios de Programación Orientada a Objetos

Encapsulamiento

Los datos internos se consultan y modifican mediante propiedades. El prefijo _ indica uso interno; las validaciones de los métodos son las que controlan los valores permitidos. Por ejemplo, salario rechaza cero, negativos y cantidades no finitas antes de modificar _salario. El mismo control se aplica al crear al empleado y al cambiar posteriormente su salario.

```
@property
def salario(self) -> float:
    """Lee el salario mediante el acceso público previsto."""
    return self._salario

@salario.setter
def salario(self, valor: float):
    """Impide salarios cero, negativos y no finitos."""
    self._salario = positivo(valor, 'Salario')
```

Herencia y Reutilización

Empleado hereda de Persona y utiliza super().__init__ para inicializar los datos personales sin repetir sus asignaciones y validaciones. La responsabilidad de gerencia se maneja con el atributo rol: permite cambiar la función del trabajador conservando su identidad y sus datos laborales. La asignación como responsable comprueba además que pertenezca al departamento que dirige.

Los repositorios comparten BaseDatos para abrir, confirmar, revertir y cerrar conexiones. Las validaciones del dominio se reutilizan al construir y recuperar los objetos. Las operaciones CRUD quedan explícitas en cada repositorio, de acuerdo con los datos y reglas de su entidad.

Abstracción y Polimorfismo

Informe establece el método abstracto exportar. InformeNomina lo implementa para sumar salarios e InformeProyecto para sumar horas. El menú recorre una colección de estos objetos e invoca el mismo método, sin decidir el comportamiento mediante condiciones sobre su clase.

```
def mostrar_informes(sistema):
    """Procesa una lista mixta usando el mismo método, sin condicionales por tipo."""
    informes = [InformeNomina(sistema.empleados.listar())]
    registros = sistema.registros.listar()
    for proyecto in sistema.proyectos.listar():
        informes.append(InformeProyecto(proyecto, registros))
    for informe in informes:
        print(informe.exportar())
```

Los resúmenes utilizan los datos que administra el sistema. El informe de nómina muestra la suma de salarios registrados; no corresponde a una liquidación legal de remuneraciones.

4. Persistencia y Acceso a los Datos

4.1 Conexión y Configuración

Se utiliza SQLite porque permite almacenar el sistema en un archivo local y ejecutar la aplicación sin un servidor adicional. La conexión se realiza con sqlite3, módulo oficial incluido en Python (Python Software Foundation, s. f.-b).

Elemento	Configuración	Uso
Motor y biblioteca	SQLite y sqlite3	Almacenamiento relacional local.
Ruta de la base	DB_NOMBRE=datos/ecotech.db	Ubicación del archivo de datos.
Archivo de ejemplo	.env.example	Formato de configuración incluido en la entrega.
Archivos locales	.env, datos y .venv	Excluidos del paquete y del seguimiento mediante .gitignore.

La configuración busca primero DB_NOMBRE en el entorno. Si no está definido, lee esa clave del archivo .env local. Las rutas relativas se resuelven desde la carpeta del proyecto. Como SQLite trabaja con un archivo, esta conexión no utiliza credenciales de servidor.

BaseDatos centraliza la apertura y el cierre. Al iniciar una operación activa las claves foráneas y establece un tiempo de espera de cinco segundos ante bloqueos. Al finalizar confirma los cambios o los revierte cuando ocurre un error.

4.2 Estructura Relacional

La base contiene cinco tablas: departamento, empleado, proyecto, empleado_proyecto y registro_tiempo. Empleado almacena también los datos heredados de Persona. La tabla intermedia empleado_proyecto representa la participación de varios empleados en varios proyectos.

```
CREATE TABLE IF NOT EXISTS departamento (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    nombre TEXT NOT NULL UNIQUE CHECK(length(trim(nombre)) > 0),
    gerente_id INTEGER UNIQUE REFERENCES empleado(id) ON DELETE RESTRICT
);

-- Datos heredados: los datos personales no se duplican en tablas hijas.
CREATE TABLE IF NOT EXISTS empleado (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    nombre TEXT NOT NULL CHECK(length(trim(nombre)) > 0),
    direccion TEXT NOT NULL,
    telefono TEXT NOT NULL,
    email TEXT NOT NULL UNIQUE COLLATE NOCASE,
    fecha_inicio TEXT NOT NULL,
    salario REAL NOT NULL CHECK(salario > 0),
    departamento_id INTEGER REFERENCES departamento(id) ON DELETE RESTRICT,
    rol TEXT NOT NULL CHECK(rol IN ('empleado', 'gerente'))
);

CREATE TABLE IF NOT EXISTS proyecto (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    nombre TEXT NOT NULL UNIQUE CHECK(length(trim(nombre)) > 0),
    descripcion TEXT NOT NULL,
    fecha_inicio TEXT NOT NULL
);

CREATE TABLE IF NOT EXISTS empleado_proyecto (
    empleado_id INTEGER NOT NULL REFERENCES empleado(id) ON DELETE CASCADE,
    proyecto_id INTEGER NOT NULL REFERENCES proyecto(id) ON DELETE CASCADE,
    PRIMARY KEY(empleado_id, proyecto_id)
);

CREATE TABLE IF NOT EXISTS registro_tiempo (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    empleado_id INTEGER NOT NULL,
    proyecto_id INTEGER NOT NULL,
    fecha TEXT NOT NULL,
    horas REAL NOT NULL CHECK(horas > 0 AND horas <= 24),
    descripcion TEXT NOT NULL,
    FOREIGN KEY(empleado_id, proyecto_id)
        REFERENCES empleado_proyecto(empleado_id, proyecto_id) ON DELETE RESTRICT
);
```

Las claves primarias identifican los registros y las claves foráneas comprueban las relaciones. Una asignación con horas registradas no puede eliminarse sin resolver primero sus registros dependientes. Esto evita perder el vínculo necesario para interpretar el historial de trabajo.

Las fechas se almacenan como texto AAAA-MM-DD y se convierten a date al reconstruir los objetos. Los salarios se almacenan como REAL. Se aplica además la regla de que un empleado no registre más de 24 horas en un día, sumando todos sus proyectos. RegistroTiempoRepositorio consulta el total previo y comprueba la suma antes de guardar. EmpleadoRepositorio y DdepartamentoRepositorio revisan la responsabilidad del área mediante consultas y condiciones de Python. Las claves y restricciones relacionales permanecen en SQLite.

PRAGMA foreign_keys = ON se ejecuta en cada conexión para aplicar la integridad referencial definida en el esquema (SQLite, s. f.).

4.3 Operaciones CRUD

Se implementaron operaciones de creación, consulta, actualización y eliminación para Empleado, Departamento, Proyecto y RegistroTiempo. Cada repositorio declara sus consultas completas y entrega por separado los valores externos mediante parámetros. La tabla siguiente muestra el caso de Departamento.

Operación	Método	Sentencia SQL
Crear	guardar	INSERT INTO departamento (nombre, gerente_id) VALUES (?, ?)
Consultar	obtener	SELECT id, nombre, gerente_id FROM departamento WHERE id = ?
Listar	listar	SELECT id, nombre, gerente_id FROM departamento ORDER BY id
Actualizar	actualizar	UPDATE departamento SET nombre = ?, gerente_id = ? WHERE id = ?
Eliminar	eliminar	DELETE FROM departamento WHERE id = ?

Empleado utiliza el mismo flujo con sus columnas y valores. Las consultas de lectura sin filtros externos, como listar, se ejecutan directamente; las operaciones que reciben datos utilizan los marcadores correspondientes.

Operación	Sentencia SQL de Empleado
Crear	INSERT INTO empleado (nombre, direccion, telefono, email, fecha_inicio, salario, departamento_id, rol) VALUES (?, ?, ?, ?, ?, ?, ?, ?)
Consultar	SELECT id, nombre, direccion, telefono, email, fecha_inicio, salario, departamento_id, rol FROM empleado WHERE id = ?
Listar	SELECT id, nombre, direccion, telefono, email, fecha_inicio, salario, departamento_id, rol FROM empleado ORDER BY id
Actualizar	UPDATE empleado SET nombre = ?, direccion = ?, telefono = ?, email = ?, fecha_inicio = ?, salario = ?, departamento_id = ?, rol = ? WHERE id = ?
Eliminar	DELETE FROM empleado WHERE id = ?

EmpleadoRepositorio ejecuta directamente estas operaciones. guardar_en_conexion realiza la inserción sin confirmar por separado, lo que permite utilizarla dentro de una transacción que incluya otras escrituras. El ID se incorpora al objeto después de confirmar el registro.

```
def guardar_en_conexion(self, conn, objeto: Empleado) -> int:
    """Inserta usando la conexión recibida, sin confirmar por separado."""
    if objeto.id is not None:
        raise DatoInvalidoError('El objeto ya tiene ID; utiliza actualizar.')
    valores = self.valores(objeto)
    cursor = conn.execute(self.SQL_INSERTAR, valores)
    return cursor.lastrowid


def guardar(self, objeto: Empleado):
    """Asigna el ID al objeto después de confirmar la inserción."""
    with self.base.conexion() as conn:
        nuevo_id = self.guardar_en_conexion(conn, objeto)
    objeto.id = nuevo_id
    return objeto


def obtener(self, id: int):
    """Consulta un ID; devuelve un objeto o None si no existe."""
    identificador(id)
    with self.base.conexion() as conn:
        fila = conn.execute(self.SQL_OBTENER, (id,)).fetchone()
    if fila is None:
        return None
    return self._a_objeto(fila)


def actualizar(self, objeto):
    """Guarda los atributos de una entidad existente."""
    identificador(objeto.id)
    valores = self.valores(objeto)
    with self.base.conexion() as conn:
        self._validar_responsabilidad(conn, objeto)
        cursor = conn.execute(self.SQL_ACTUALIZAR, valores + (objeto.id,))
        if cursor.rowcount == 0:
            raise RegistroNoEncontradoError('No existe el registro que quieres editar.')
    return objeto


def eliminar(self, id: int):
    """Elimina una fila si sus relaciones lo permiten."""
    identificador(id)
    with self.base.conexion() as conn:
        cursor = conn.execute(self.SQL_ELIMINAR, (id,))
        if cursor.rowcount == 0:
            raise RegistroNoEncontradoError('No existe el registro que quieres eliminar.')
```

4.4 Conversión de Filas en Objetos

Una consulta devuelve filas con valores ordenados según las columnas del SELECT. El método _a_objeto convierte esa tupla en una instancia de la entidad. Consultar, listar y buscar utilizan el mismo mapeo, lo que evita reconstruir de manera diferente un mismo tipo de registro.

```
def _a_objeto(self, fila) -> Empleado:
    """Mapper centralizado: reconstruye y valida una fila."""
    return Empleado(
        nombre=fila[1],
        direccion=fila[2],
        telefono=fila[3],
        email=fila[4],
        fecha_inicio=fila[5],
        salario=fila[6],
        departamento_id=fila[7],
        rol=fila[8],
        id=fila[0],
    )
```

Al construir Empleado se recuperan también los datos de Persona y se vuelven a aplicar las validaciones del dominio. El mapeo debe conservar el orden de columnas declarado en las consultas.

4.5 Comprobación de Consultas Parametrizadas

Se probó una búsqueda con el texto de ataque indicado a continuación. La consulta y el dato se envían separados a SQLite, por lo que las comillas y los operadores recibidos se interpretan como parte del texto buscado. No se incorporan a la estructura de la instrucción SQL.

```
def buscar(self, texto: str) -> list:
    """El texto se envía como parámetro separado de la consulta."""
    with self.base.conexion() as conn:
        filas = conn.execute(self.SQL_BUSCAR, (texto,)).fetchall()
    objetos = []
    for fila in filas:
        objetos.append(self._a_objeto(fila))
    return objetos


# Fragmento de probar_sql en demo.py:
ataque = "' OR '1'='1"
resultados = sistema.empleados.buscar(ataque)
assert resultados == []
```

En los datos preparados había dos empleados y ninguno coincidía con ese texto. La búsqueda devolvió una lista vacía y las filas originales se conservaron. La demostración puede repetirse con python demo.py --caso sql; su registro está en evidencias/sql.txt.

```
PRUEBA DE CONSULTA PARAMETRIZADA
Empleados existentes: 2
Consulta SQL real:
SELECT id, nombre, direccion, telefono, email, fecha_inicio, salario, departamento_id, rol FROM empleado WHERE
instr(lower(nombre), lower(?)) > 0 ORDER BY id
Parámetro enviado por separado: "' OR '1'='1"
Resultado de buscar el payload: []
Filas devueltas: 0
COMPROBADO: la entrada se interpreta como texto, no como SQL.
```

Figura 2. Resultado de la búsqueda parametrizada con el texto de ataque.

5. Manejo de Errores y Transacciones

5.1 Excepciones del Sistema

Se definió EcoTechError como excepción base y se distinguieron los errores de datos, duplicados, relaciones, registros inexistentes y persistencia. Esta clasificación permite informar el problema de acuerdo con la operación realizada, sin mostrar directamente los mensajes internos de SQLite.

```
"""Errores que el menú puede explicar sin mostrar detalles de SQLite."""


class EcoTechError(Exception):
    """Error controlado del sistema."""


class DatoInvalidoError(EcoTechError, ValueError):
    """Un dato incumple una regla del dominio."""


class RegistroNoEncontradoError(EcoTechError):
    """No existe el registro solicitado."""


class DuplicadoError(EcoTechError):
    """El correo, nombre o asignación ya está registrado."""


class RelacionError(EcoTechError):
    """Una operación dejaría relaciones inconsistentes."""


class PersistenciaError(EcoTechError):
    """La base o su configuración no están disponibles."""
```

La conexión captura sqlite3.IntegrityError. Cuando la causa corresponde a una restricción de unicidad, genera DuplicadoError; para otras restricciones genera RelacionError. Las fallas de conexión se comunican mediante PersistenciaError. raise ... from error conserva la causa original para revisar el problema.

5.2 Validación de Entradas y Control de Errores

El menú convierte y valida las entradas antes de operar. Se revisan texto obligatorio, correo, fechas, salario, rol, horas e identificadores. El ID debe ser un entero positivo dentro del rango admitido por SQLite; esta condición evita enviar al motor valores que no puede procesar.

```
def identificador(valor: int, opcional: bool = False) -> int:
    """Impide IDs nulos obligatorios y enteros fuera del rango de SQLite."""
    if valor is None and opcional:
        return None
    if type(valor) is not int or not 1 <= valor <= MAX_ID:
        raise DatoInvalidoError('El ID debe estar entre 1 y 9223372036854775807.')
    return valor
```

Las operaciones del menú capturan las excepciones previstas y muestran un mensaje que permite reconocer el problema. Por ejemplo, contratar informa si el empleado o la asignación no pudieron registrarse.

```
def contratar(sistema):
    """Expone en consola una operación real de dos escrituras atómicas."""
    try:
        print('Crear empleado y asignarlo a un proyecto existente en una sola operación.')
        empleado = formulario('empleados')
        proyecto_id = leer_id('ID proyecto: ')
        sistema.contratar_en_proyecto(empleado, proyecto_id)
        print('Empleado y asignación guardados con ID', empleado.id)
    except (ValueError, EcoTechError) as error:
        print('No se guardó la contratación:', error)
```

Los errores no se silencian con except: pass. Ante una excepción inesperada, el administrador de conexión revierte los cambios y vuelve a lanzarla; finally asegura el cierre de la conexión.

5.3 Confirmación y Reversión de Operaciones

La contratación en un proyecto requiere guardar al empleado y su asignación. Ambas escrituras utilizan la misma conexión y se confirman juntas. Si la segunda falla, rollback revierte la primera para evitar que la operación quede incompleta.

```
def contratar_en_proyecto(self, empleado: Empleado, proyecto_id: int):
    """Inserta empleado y asignación en la misma transacción, todo o nada."""
    identificador(proyecto_id)
    with self.base.conexion() as conn:
        nuevo_id = self.empleados.guardar_en_conexion(conn, empleado)
        conn.execute(
            'INSERT INTO empleado_proyecto (empleado_id, proyecto_id) VALUES (?, ?)',
            (nuevo_id, proyecto_id),
        )
    # Si falla la segunda escritura, no se asigna al objeto un ID inexistente.
    empleado.id = nuevo_id
    return empleado
```

El administrador de conexión inicia la transacción con BEGIN. Las consultas de validación y las escrituras de una operación quedan dentro de esa misma transacción. commit confirma al terminar; las capturas realizan rollback ante una falla y finally cierra el recurso.

```
@contextmanager
def conexion(self):
    """Confirma al terminar; si falla, revierte y traduce el error."""
    conn = None
    try:
        conn = sqlite3.connect(str(self.ruta), timeout=5)
        conn.execute('PRAGMA foreign_keys = ON')
        conn.execute('BEGIN')
        yield conn
        conn.commit()
    except sqlite3.IntegrityError as error:
        if conn:
            conn.rollback()
        if 'UNIQUE' in str(error):
            raise DuplicadoError('Ese correo, nombre o asignación ya existe.') from error
        raise RelacionError(
            'Operación rechazada: revisa relaciones, responsable y total diario de horas.'
        ) from error
    except (sqlite3.Error, OSError) as error:
        if conn:
            conn.rollback()
        raise PersistenciaError(
            'No se pudo operar con la base. Revisa ruta y permisos.'
        ) from error
    except Exception:
        if conn:
            conn.rollback()
        raise
    finally:
        if conn:
            conn.close()
```

La prueba de reversión intenta asignar al empleado al proyecto 999, que no existe. Antes de la operación no hay empleados; después del error tampoco quedan empleados ni asignaciones, y el objeto

conserva id=None. También se comprobó el caso de un proyecto válido, en el que ambas escrituras quedan guardadas.

```
PRUEBA DE TRANSACCIÓN ATÓMICA
Empleados antes: 0
Primera escritura: INSERT del empleado.
Segunda escritura: INSERT de asignación al proyecto inexistente 999.
Error capturado: RelacionError
Operación rechazada: revisa relaciones, responsable y total diario de horas.
Empleados después del rollback: 0
ID del objeto tras el fallo: None
Asignaciones después: []
COMPROBADO: la primera escritura también se revirtió.
```

Figura 3. Estado de los datos después del fallo en la segunda escritura.

6. Uso de IA y Revisión del Código

6.1 Herramienta y Solicitudes Técnicas

Se utilizó Codex como herramienta de apoyo en la generación y corrección de código, la documentación y las comprobaciones técnicas. Las propuestas se revisaron según su funcionamiento, la organización de responsabilidades y la protección de los datos.

Herramienta y fecha	Solicitud	Resultado
Codex · 19/10/2026	Genera una versión orientativa de lo que vamos a trabajar	Borrador de código bastante complejo. Se uso en un inicio, pero se descarto parcialmente al no comprenderlo
Codex · 19/10/2026	Proponer la implementación modular del sistema.	Borrador de clases en dominio/, persistencia en repositorio/ y menú en main.py.
Codex · 20/10/2026	Revisar consultas, validaciones y relaciones.	Correcciones y comprobaciones en los repositorios, validaciones.py, tests/test_sistema.py y demo.py.
Codex · 22/10/2026	Simplificar el código conservando su funcionamiento.	Formularios explícitos, validación del correo con condiciones y mapeo de tuplas.

6.2 Análisis de Propuestas y Decisiones

La revisión consideró fragmentos concretos y su comportamiento. Se modificaron las propuestas que dificultaban el control de los datos o su comprensión, y se conservaron aquellas cuyo resultado pudo comprobarse. Los fragmentos finales, las decisiones y las comprobaciones están en docs/REVISION_TECNICA_IA.md.

Fragmento revisado	Decisión	Justificación y comprobación
Consultas de inserción y búsqueda	Modifico	Se sustituyeron por sentencias completas y fijas. Los valores ya estaban parametrizados; la modificación elimina la construcción dinámica de SQL. Se conserva la prueba de búsqueda con texto de ataque.

Fragmento revisado	Decisión	Justificación y comprobación
validaciones.py: identificador	Modifico	El límite superior evita enviar a SQLite enteros que producen OverflowError. Las pruebas verifican el rechazo y la continuidad del menú.
Organización de las operaciones CRUD	Modifico	Se organizaron las operaciones completas por entidad, siguiendo los ejemplos de clase para facilitar su lectura. Se comparten conexión y validaciones. Los cuatro ciclos CRUD y el mapeo siguen funcionando.
Validación de horas y responsables	Modifico	Las reglas se expresaron con SELECT y condiciones de Python en los repositorios. Se conservan claves foráneas y transacciones. Las pruebas verifican el máximo diario, su edición y la pertenencia del responsable.
Validación de una fecha editada	Corrijo	Una prueba nueva detectó que una fecha recibida como texto no admitía isoformat directamente. Se aplicó fecha antes de convertirla. La edición hacia un día completo se rechaza y conserva el dato anterior.
BaseDatos.conexion y la contratación	Acepto y ajusto	Se mantuvo commit, rollback y cierre, y se agregó BEGIN antes de las comprobaciones. La operación de dos escrituras confirma ambas o revierte todo; el ID se asigna después de confirmar.
Gerencia mediante Empleado.rol	Acepto recomendación de IA	La IA recomendó representar la gerencia como rol, siguiendo C06. Esta adaptación de la subclase Gerente del ES1 permite cambiar la función sin duplicar al trabajador. Se comprueban la conservación del ID, el rol y la pertenencia al departamento.

6.3 Resultados de las Comprobaciones

Se ejecutaron 37 pruebas automáticas con asistencia de IA. Los casos cubren el CRUD de las cuatro entidades, las relaciones, las validaciones, la búsqueda parametrizada, los formularios, el mapeo y las transacciones. Los resultados se conservan en evidencias/pruebas.txt y evidencias/revision_tecnica.txt.

7. Trabajo Individual y Conclusiones

7.1 Aportes y Responsabilidad de la Entrega

La entrega reúne el código fuente, el esquema de la base de datos, la documentación y las evidencias de funcionamiento. El apoyo técnico de IA utilizado durante el desarrollo y la revisión se identifica en la sección 6.

Responsable	Participación	Responsabilidad
Benjamin Olivares Mella RUT: 20.777.250-K	Organización del trabajo, materiales y criterios del proyecto, y solicitud de desarrollo y revisión técnica con apoyo de IA.	Entrega individual. Firma: _____

7.2 Conclusiones

8. Referencias Bibliográficas

INACAP. (2026a). Guía de aprendizaje de la Evaluación Sumativa de la Unidad 2, TI3021 [Material de asignatura].

INACAP. (2026b). Los pilares en Python; Persistencia segura; CRUD parametrizado [Materiales de clase].

Olivares Mella, B. (2026). Informe técnico ES1: Modelado orientado a objetos. Caso EcoTech Solutions [Informe de asignatura].

Python Software Foundation. (s. f.-a). abc — Abstract Base Classes. Python documentation. https://docs.python.org/3/library/abc.html

Python Software Foundation. (s. f.-b). sqlite3 — DB-API 2.0 interface for SQLite databases. Python documentation. https://docs.python.org/3/library/sqlite3.html

SQLite. (s. f.). SQLite Foreign Key Support. https://www.sqlite.org/foreignkeys.html